Web Applications Development Assignment 2026

Submission Deadline: 15/06@23:59

Objective

In this assignment, you are required to implement a web application that allows users to:

- Explore the MovieLens Latest Small dataset
- Add new movies
- Rate movies
- Obtain personalized movie recommendations

Details about the dataset can be found at:
`https://grouplens.org/datasets/movielens/latest/`

Backend

Dataset and Database

You must download the dataset from:
`https://files.grouplens.org/datasets/movielens/ml-latest-small.zip`
and load it into an SQLite database.
The database must include the following tables:

- `movies`
- `ratings`
- `tags`

Each table must:

- Follow the structure of the corresponding CSV file
- Be initially populated with the dataset contents

API Specification

Assuming a base URL:

`http://{your-server-domain}:3000/movielens/api`

you must implement the following RESTful APIs using **Python FastAPI**.

1. Search Movies

```
GET /movies?search={keyword}
```

Description: Returns all movies whose title contains the given keyword (case-insensitive).
Response:

```
{
  "status": "success",
  "movies": [ ... ]
}
```

2. Get Ratings for a Movie

```
GET /ratings/{movieId}
```

Description: Returns all ratings for the specified movie.
Response:

```
{
  "status": "success",
  "ratings": [ ... ]
}
```

3. Add a New Movie

```
POST /movies
```

Request Body:

```
{
  "title": "Movie Title",
  "genres": "Action|Drama"
}
```

Response:

```
{
  "status": "success",
  "movieId": 12345
}
```

4. Get Recommendations

```
POST /recommendations
```

Description: Returns movie recommendations based on the ratings provided by the web application user.
Request Body:

```
{
  "ratings": [
    { "movieId": 1, "rating": 4.5 },
    { "movieId": 32, "rating": 5.0 }
  ]
}
```

Algorithm:

1. For the web application user u, identify users v with overlapping rated movies.
2. Compute similarity:

$$sim(u, v) = \text{Pearson correlation on co-rated items}$$

3. Select the top-K most similar users.
4. For each candidate movie i (not rated by u), predict:

$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_{v \in N(u)} sim(u, v) \cdot (r_{v,i} - \bar{r}_v)}{\sum_{v \in N(u)} |sim(u, v)|}$$

5. Recommend the top-N movies with the highest predicted ratings.

Response:

```
{
  "status": "success",
  "recommendations": [
    {
      "movieId": 296,
      "title": "...",
      "genres": "...",
      "predictedRating": 4.72
    }, ...
  ]
}
```

Remarks

- New records must be assigned unique IDs.
- The backend must enable CORS.
- The ratings provided in the recommendation request are used only for that request and do not need to be stored.
- You may choose appropriate values for K and N.

Frontend

The frontend must be implemented using:

- HTML
- CSS
- Vanilla JavaScript

Constraints:

- One `index.html` file
- One `index.js` file
- One `index.css` file
- No frameworks or external libraries are allowed

Required Functionality

- Add a new movie (with success/failure feedback)
- Search movies by keyword
- Rate movies (ratings stored only in browser memory during the session)
- Display average rating of a movie
- Request and display recommendations

UI Requirements

- Clear and intuitive interface
- Proper user guidance (labels, instructions)
- Meaningful error messages
- Structured presentation (e.g., tables)

Deliverables

Submit a compressed file named:

`XXXX.zip`

(where `XXXXX` are the last five digits of your student ID)

Frontend

A directory named "frontend" with the following files:

- `index.html`
- `index.js`
- `index.css`

Backend

A directory named "backend" with the following files:

- All source code files
- `requirements.txt` (or equivalent dependency manifest)
- `README.md` with setup and execution instructions
- Script(s) to create and populate the SQLite database
- Any configuration files required to run the backend

Miscellaneous

- You may use AI tools, but you must be able to explain your code.
- You must be able to demonstrate and extend your application.
- Late submissions will be penalized: -5 per day counting from the first second.